

TP Avancé : Conception, Déploiement et Sécurisation d'Architectures Web Modernes avec Supabase

Objectifs pédagogiques

L'objectif de ce TP est de permettre aux étudiants d'expérimenter différentes architectures web (Client-Serveur, Microservices, et DApps) tout en intégrant des notions de **sécurité**, de **scalabilité**, et de **décentralisation**.

À la fin du TP, vous serez capables de : - Concevoir une application web selon trois architectures différentes. - Mettre en œuvre une authentification sécurisée via **Supabase**. - Identifier les failles de sécurité possibles et proposer des mesures correctives. - Comparer les performances et la robustesse de chaque approche.

Contexte

L'entreprise fictive **DataSecure Corp** souhaite lancer un service web baptisé **SafeDocs**, permettant à ses utilisateurs de stocker et gérer leurs documents de manière sécurisée.

L'équipe technique doit explorer trois architectures possibles : 1. Une version **client-serveur classique**. 2. Une version **modulaire en microservices**. 3. Une version **décentralisée (DApp)** basée sur la blockchain.

Chaque équipe devra concevoir un prototype pour ces trois versions, analyser leur fonctionnement et identifier leurs points forts et limites.

Durée

6 heures (réparties en deux séances de 3h)

Partie 1 – Architecture Client-Serveur avec Supabase

Objectif

Mettre en place une application web simple (par exemple un mini gestionnaire de fichiers) basée sur une architecture **client-serveur**.

Étapes

1. Créez un projet **Supabase** (<https://supabase.com/>).

2. Activez les modules suivants :
 - **Authentication** : pour gérer les inscriptions et connexions.
 - **Database (PostgreSQL)** : pour stocker les utilisateurs et les fichiers.
 - **Storage** : pour héberger les fichiers des utilisateurs.
3. Développez un **frontend** en React, Vue.js ou Svelte :
 - Authentification avec Supabase Auth.
 - Interface d'upload de fichier.
 - Liste des fichiers personnels de l'utilisateur connecté.

Sécurité à vérifier

- Respect du modèle **RLS (Row Level Security)** dans Supabase.
- Configuration correcte des **politiques** (chaque utilisateur ne voit que ses fichiers).
- Vérification du CORS entre le front et Supabase.

Outils recommandés

- **VS Code, Supabase CLI, Postman.**
- Optionnel : **OWASP ZAP** pour tester la sécurité des endpoints.

Partie 2 – Architecture Microservices avec API et Supabase

Objectif

Transformer l'application précédente en **architecture microservices**.

Étapes

1. Créez **3 services distincts** :
 - **auth-service** : gère les utilisateurs (en s'appuyant sur Supabase Auth).
 - **file-service** : gère le stockage et les métadonnées des fichiers (via Supabase Storage et Database).
 - **api-gateway** : unique point d'entrée vers les autres services.
2. Chaque service doit :
 - Être conteneurisé avec **Docker**.
 - Communiquer via **API REST**.
 - Avoir sa propre logique et configuration.
3. Déployez les services localement via **Docker Compose**.

Sécurité à tester

- Protection de chaque endpoint avec token JWT.

- Communication HTTPS (via un reverse proxy Nginx).
- Attaques possibles entre services internes.

Outils recommandés

- Docker, Docker Compose, Postman, Wireshark, OpenSSL.
-

Partie 3 – Architecture Décentralisée (DApp)

Objectif

Développer une version décentralisée de SafeDocs où les documents sont enregistrés sur **IPFS** et les utilisateurs gérés via un **smart contract**.

Étapes

1. Écrire un **smart contract** `SafeDocs.sol` qui :
 - Enregistre les utilisateurs.
 - Associe leurs fichiers (liens IPFS).
 - Permet de consulter ses fichiers en lecture.
2. Déployer le contrat sur un **testnet Ethereum (Sepolia)** ou **Polygon Mumbai**.
3. Créer une **interface React** :
 - Connexion via **MetaMask**.
 - Interaction avec le contrat via **Ethers.js**.
 - Affichage des fichiers stockés sur IPFS.

Sécurité à vérifier

- Risques liés au stockage IPFS (liens publics).
- Gestion des clés privées (MetaMask).
- Tests sur la résistance du smart contract (reentrancy, gas limit).

Outils recommandés

- Remix IDE, Hardhat, MetaMask, Ethers.js, IPFS Desktop.
-

Partie 4 – Rapport d’analyse

Chaque groupe doit rédiger un **rapport technique** structuré :

Partie	Contenu attendu
Introduction	Présentation de SafeDocs et des trois architectures
Développement	Description technique et schémas d’architecture

Partie	Contenu attendu
Sécurité	Tests réalisés et failles détectées
Comparaison	Tableau comparatif des performances, sécurité, et maintenance
Conclusion	Architecture la plus adaptée selon le contexte

Bonus (facultatif)

- Ajouter un **reverse proxy Nginx** avec HTTPS.
- Mettre en place un **tableau de bord Grafana** pour surveiller les microservices.
- Intégrer un **audit automatisé de sécurité** (via GitHub Actions ou OWASP CI).
- Simuler une **attaque XSS** sur le front et proposer une correction.

Évaluation

Critère	Pondération
Fonctionnalités opérationnelles	25%
Sécurité et conformité	25%
Qualité de l'architecture et du code	20%
Rapport technique	20%
Originalité et exploration	10%

Concepts et Technologies – Explications

Supabase

Supabase est une plateforme open source qui fournit : - une **base de données PostgreSQL** prête à l'emploi,
- une **authentification intégrée**,
- une **API REST automatique**,
- et un **stockage de fichiers** sécurisé.

Elle permet de créer rapidement un backend complet sans avoir à coder toute l'infrastructure serveur.

Microservices

Les microservices consistent à découper une application en plusieurs petits programmes autonomes.

Chaque service se charge d'une seule tâche (authentification, stockage, API, etc.).

Ce découpage permet une **meilleure scalabilité**, mais rend la communication entre services plus complexe.

API Gateway

L'API Gateway est un point d'entrée unique pour tous les microservices.

Elle gère la redirection des requêtes, l'authentification, la sécurité, et parfois la mise en cache des réponses.

DApp (Application Décentralisée)

Une DApp fonctionne sur la **blockchain**, sans serveur central.

Les règles sont gérées par un **smart contract** déployé sur la blockchain.

Cela apporte transparence et sécurité, mais rend les mises à jour et les transactions plus coûteuses.

IPFS (InterPlanetary File System)

C'est un système de fichiers décentralisé.

Les fichiers sont stockés sur plusieurs nœuds du réseau.

Chaque fichier possède un identifiant unique (hash), et il n'existe pas de serveur unique.

Sécurité à retenir

- **CORS** : protège contre les requêtes non autorisées entre domaines.
 - **JWT** : jeton d'authentification utilisé pour sécuriser les API.
 - **HTTPS / mTLS** : chiffrement des communications.
 - **Reentrancy Attack** : vulnérabilité fréquente dans les smart contracts mal conçus.
-

Conclusion

Ce TP vise à offrir une **vision complète des architectures web modernes**, en combinant des aspects **techniques, sécuritaires** et **conceptuels**.

L'approche par Supabase permet de gagner du temps sur l'infrastructure tout en expérimentant des architectures avancées, du client-serveur classique à la décentralisation complète.
